

MICROCONTROLLERS

Labo 7

Naslag

Gegenereerd op 29/07/2026 uit tdmts.github.io/Microcontrollers

Inhoud

Onderbreken in plaats van wachten

[Van pollen naar interrupts](#)

[attachInterrupt en de interruptpinnen](#)

Werken met een ISR

[De regels van een ISR](#)

[volatile en de vlagvariabele](#)

[Denderen bij een interrupt](#)

Datasheets

[PCF8574 I/O document](#)

NASLAG

Van pollen naar interrupts

Tot nu toe las je je knoppen uit door er in je `loop()` naar te kijken. Dat werkt goed zolang je programma verder niets te doen heeft. Zodra er ook iets anders moet gebeuren, mis je drukken, en een interrupt is wat dat oplost.

Pollen: telkens opnieuw gaan kijken

Een knop uitlezen doe je met `digitalRead()`. Die functie vertelt je hoe de pin er op dat *ene moment* bij ligt. Wil je weten of er ondertussen op de knop gedrukt is, dan moet je dus blijven kijken, ronde na ronde.

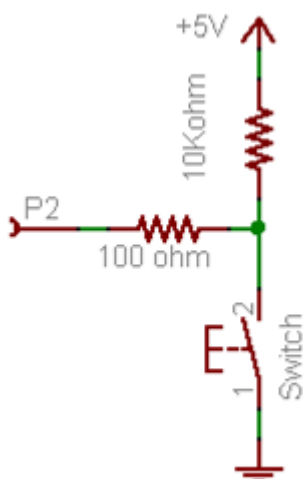
Die manier van werken heet **pollen**. Je vraagt herhaaldelijk "en nu? en nu? en nu?".

```

1  const int knopPin = 2;
2  const int ledPin = 8;
3
4  void setup()
5  {
6      pinMode(knopPin, INPUT_PULLUP);
7      pinMode(ledPin, OUTPUT);
8  }
9
10 void loop()
11 {
12     if (digitalRead(knopPin) == LOW) // ingedrukt
13     {
14         digitalWrite(ledPin, HIGH);
15     }
16 }

```

Deze `loop()` draait duizenden keren per seconde. Je kan de knop onmogelijk zo kort indrukken dat hij hem mist. Zolang dit het enige is wat je programma doet, is pollen in orde.



Een drukknop met een externe pull-upweerstand.

Gebruik je

`INPUT_PULLUP`

, dan zit die weerstand al in de chip en heb je hem niet nodig.

Waar het misgaat

Nu zet je er één regel bij.

```
1  const int knopPin = 2;
2  const int ledPin = 8;
3
4  void setup()
5  {
6      pinMode(knopPin, INPUT_PULLUP);
7      pinMode(ledPin, OUTPUT);
8  }
9
10 void loop()
11 {
12     if (digitalRead(knopPin) == LOW)
13     {
14         digitalWrite(ledPin, HIGH);
15     }
16
17     delay(1000); // hier gebeurt iets anders, wat dan ook
18 }
```

Vanaf nu wordt je knop nog maar **één keer per seconde** bekeken. Druk je de knop in en laat je hem los terwijl die `delay()` loopt, dan heeft je programma er nooit iets van gemerkt. De druk is weg.

Een kortere `delay()` lost dat niet op. Elke seconde die je programma ergens anders mee bezig is, is een seconde waarin het je knop niet bekijkt.

Een noodstop die tien seconden te laat komt

Kijk eens naar dit programma. Twee leds wisselen elkaar af, met tien seconden ertussen. Op pin 2 zit een noodstop. Druk je die in, dan moeten alle leds doven en blijven ze uit.

```
1  const int knopPin = 2;
2  const int ledPin1 = 3;
3  const int ledPin2 = 4;
4
5  bool noodstop = false;
6  int teller = 0;
7
8  void setup()
9  {
10     pinMode(knopPin, INPUT_PULLUP);
11     pinMode(ledPin1, OUTPUT);
12     pinMode(ledPin2, OUTPUT);
13 }
14
15 void loop()
16 {
17     if (digitalRead(knopPin) == LOW) // noodstop ingedrukt
18     {
19         noodstop = true;
20     }
21
22     if (noodstop == true)
23     {
24         digitalWrite(ledPin1, LOW);
25         digitalWrite(ledPin2, LOW);
26     }
27     else
28     {
29         if (teller % 2 == 0)
30         {
31             digitalWrite(ledPin1, HIGH);
32             digitalWrite(ledPin2, LOW);
33         }
34         else
35         {
36             digitalWrite(ledPin1, LOW);
37             digitalWrite(ledPin2, HIGH);
38         }
39
40         delay(10000);
41         teller++;
42     }
43 }
```

⚠ HOE LANG MOET JE DIE NOODSTOP INGEDRUKT HOUDEN VOOR DE LEDS DOVEN?

Denk er even over na voor je verder leest. Het antwoord is niet "een fractie van een seconde".

De `loop()` zit in `delay(10000)`. Pas als die tien seconden om zijn, kijkt het programma weer naar de knop. Laat je de noodstop vóór dat moment los, dan is er niets gebeurd. Je moet hem dus in het slechtste geval **tien seconden lang ingedrukt houden**.

Zo'n noodstop is onbruikbaar.

Wat een interrupt daaraan verandert

Je code is correct, maar het probleem is dat je programma zelf moet gaan kijken. Zolang het ergens anders mee bezig is, kan het dat niet.

Een **interrupt** draait dat om. Je vraagt niet meer of er iets gebeurd is, maar je wordt geroepen wanneer het gebeurt.

In de chip zit hardware die een paar pinnen in de gaten houdt, helemaal los van je programma. Ziet die hardware op zo'n pin de spanning veranderen, dan onthoudt ze dat meteen, en zodra de lopende instructie klaar is wordt je programma **onderbroken**. Het wordt middenin gepauzeerd en niet eerst afgewerkt.

De controller springt dan naar een functie die jij geschreven hebt. Die functie heet de **interrupt service routine**, kortweg de **ISR**. Zodra ze klaar is, gaat je programma verder waar het gebleven was, alsof er niets gebeurd is. Zat je in een `delay()`, dan telt die gewoon verder af.

i EEN INTERRUPT HOUDT DE FLANK VAST IN HARDWARE

Bij pollen kan je een korte druk missen omdat je op het verkeerde moment keek. Bij een interrupt is dat onmogelijk, want de flank wordt in hardware **vastgehouden**, ook als je programma net iets anders aan het doen was. Zelfs een druk van een duizendste van een seconde raakt niet verloren.

Hetzelfde noodstopprogramma, nu met een interrupt. Er veranderen maar twee dingen, en ze zijn allebei gemarkeerd:

```
1  const int knopPin = 2;
2  const int ledPin1 = 3;
3  const int ledPin2 = 4;
4
5  volatile bool noodstop = false; // 1: volatile, want de ISR verandert dit
6  int teller = 0;
7
8  // De interrupt service routine. Kort, en meer niet.
9  void noodstopIngedrukt()
10 {
11     digitalWrite(ledPin1, LOW);
12     digitalWrite(ledPin2, LOW);
13
14     noodstop = true;
15 }
16
17 void setup()
18 {
19     pinMode(ledPin1, OUTPUT);
20     pinMode(ledPin2, OUTPUT);
21     pinMode(knopPin, INPUT_PULLUP);
22
23     // 2: koppel de functie aan de pin
24     attachInterrupt(digitalPinToInterrupt(knopPin), noodstopIngedrukt, FALLING);
25 }
26
27 void loop()
28 {
29     if (noodstop == true)
30     {
31         digitalWrite(ledPin1, LOW);
32         digitalWrite(ledPin2, LOW);
33     }
34     else
35     {
36         if (teller % 2 == 0)
37         {
38             digitalWrite(ledPin1, HIGH);
39             digitalWrite(ledPin2, LOW);
40         }
41         else
42         {
43             digitalWrite(ledPin1, LOW);
44             digitalWrite(ledPin2, HIGH);
45         }
46
47         delay(10000);
48         teller++;
49     }
50 }
```

Merk op dat de `loop()` nu niet meer naar de knop kijkt. Die regel is verdwenen. En toch doven de leds op het moment dat je drukt, ook al zit de `loop()` nog acht seconden vast in zijn `delay()`.

Het belangrijkste idee van dit labo

Kijk goed naar wat er in die ISR staat, en vooral naar wat er niet in staat.

De ISR doet het **dringende werk aan de hardware** en zet de leds uit. Dat moet nu gebeuren, en het duurt maar een paar microseconden. Daarnaast zet ze een **vlag**: `noodstop = true`. Die vlag is er voor de `loop()`, zodat die weet dat hij de leds niet opnieuw mag aansteken zodra zijn `delay()` afgelopen is.

ONTHOU DEZE ENE ZIN

De ISR doet het dringende werk, de vlag doet het beleid.

Alles wat echt niet kan wachten en in een oogwenk klaar is, zet je in de ISR. Alles wat mag wachten tot je `loop()` weer aan de beurt is, laat je aan de `loop()` over. Bijna elke fout die je dit labo maakt, komt erop neer dat je te veel in de ISR gestopt hebt.

i ALARM 1202, TIJDENS DE DALING NAAR DE MAAN

Tijdens de daling van Apollo 11 naar de maan, op 20 juli 1969, sloeg de boordcomputer alarm: 1202. De oorzaak was een radar die op dat moment niets te doen had en door een fout voortdurend pulsen opwekte. Elke puls kwam binnen als een interrupt, en de computer sprong telkens naar de routine die ze moest tellen. Zo'n sprong duurde kort, maar samen namen ze ongeveer 13 procent van de rekentijd in beslag, en net die tijd had de computer nodig om de landing te blijven berekenen. Dat het toch goed afliep, komt doordat elk stuk werk een prioriteit had meegekregen: de computer liet zijn taken met lage prioriteit vallen en bleef sturen.

In jouw programma gaat het om diezelfde rekentijd. Elke microseconde die je in je ISR doorbrengt, is een microseconde die je `loop()` niet krijgt. Daarom staan er in de ISR hierboven twee `digitalWrite()`'s en een vlag, en verder niets.



 De DSKY, waarmee de bemanning de boordcomputer bediende. Bij een alarm lichtte links het venster PROG op, en het nummer las de bemanning af in de cijfervensters rechts. Bron: NASA

Verder lezen

- [attachInterrupt en de interruptpinnen](#) voor de twee regels uit het voorbeeld, en waarom je niet zomaar eender welke pin mag nemen.
- [De regels van een ISR](#) voor wat er wel en niet in zo'n functie mag.
- [volatile en de vlagvariabele](#) voor het woord `volatile` en het vlagpatroon.

attachInterrupt en de interruptpinnen

Een interrupt instellen is één regel code. Maar er zitten twee dingen in die je moet begrijpen, want als je ze verkeerd invult krijg je geen foutmelding en reageert je programma nooit.

De regel zelf

```
attachInterrupt(digitalPinToInterrupt(knopPin), noodstopIngedrukt, FALLING);
```

CPP

Er staan drie dingen tussen de haakjes:

Onderdeel	Wat het is
<code>digitalPinToInterrupt(knopPin)</code>	Welke pin je in de gaten wil houden. Zie hieronder waarom je die functie eromheen zet.
<code>noodstopIngedrukt</code>	Welke functie er uitgevoerd moet worden. De naam kies je zelf. Let op, er staan geen haakjes achter. Je roept de functie hier niet op, je geeft ze door.
<code>FALLING</code>	Wanneer hij moet afgaan. Zie de tabel verderop.

De drie onderdelen van attachInterrupt

Alleen pin 2 en pin 3

Dit is de belangrijkste beperking van het hele labo. Op een **Arduino UNO** kunnen maar **twee** pinnen een interrupt opwekken: **pin 2** en **pin 3**. De rest van je pinnen kan het niet.

⚠ OP EEN VERKEERDE PIN GEBEURT ER NOOIT IETS

Zet je je knop op pin 7 en schrijf je `attachInterrupt(digitalPinToInterrupt(7), ...)`, dan compileert dat probleemloos en uploadt het probleemloos. Je programma doet alleen nooit iets.

`digitalPinToInterrupt()` geeft voor een pin die het niet kan de waarde `NOT_AN_INTERRUPT` terug, en `attachInterrupt()` negeert die stilzwijgend.

Reageert je interrupt niet, kijk dan altijd eerst na of je knop wel op pin 2 of pin 3 zit.

Voor andere borden ligt het anders. Dat is handig om te weten, al werk je voor dit labo met de UNO:

Bord	Bruikbare pinnen
UNO, Nano, Mini, andere 328-borden	2, 3
Micro, Leonardo, andere 32u4-borden	0, 1, 2, 3, 7
Mega, Mega2560, MegaADK	2, 3, 18, 19, 20, 21
UNO WiFi Rev.2, Nano Every, Due, Zero	alle digitale pinnen (bij de Zero: alle behalve pin 4)

Welke pinnen een interrupt aankunnen, per bord

Waarom digitalPinToInterrupt eromheen staat

De hardware in de chip werkt met **interruptnummers** in plaats van met pinnummers, en die twee zijn niet gelijk. Op een UNO hoort pin 2 bij interrupt 0, en pin 3 bij interrupt 1.

Je zou dus `attachInterrupt(0, ...)` kunnen schrijven als je een knop op pin 2 hebt. Doe dat niet. Die 0 zegt niets, en op een ander bord betekent hij iets anders. `digitalPinToInterrupt()` doet de omzetting voor jou:

```
1  const int knopPin = 2;
2
3  // Doe dit:
4  attachInterrupt(digitalPinToInterrupt(knopPin), mijnISR, FALLING);
5
6  // En niet dit, ook al werkt het op deze ene UNO:
7  attachInterrupt(0, mijnISR, FALLING);
```

CPP

Verhuis je je knop naar pin 3, dan hoef je alleen `knopPin` aan te passen en klopt de rest vanzelf.

i DE INTERRUPTNUMMERS VAN DE LEONARDO

In de les heb je de tabel voor de Leonardo bekeken. Daar zijn er vijf interrupts, en de nummering is veel minder logisch:

Digitale pin	Interrupt nummer
0	2
1	3
2	1
3	0
7	4

 De Leonardo: pin 3 is interrupt 0, pin 2 is interrupt 1. Precies omgekeerd aan de UNO.

Kijk eens naar pin 2 en pin 3. Op de UNO is pin 2 interrupt 0; op de Leonardo is pin 3 dat. Schrijf je een ruw nummer in je code, dan doet dezelfde sketch op het andere bord dus iets compleet anders, zonder waarschuwing. Daarom bestaat `digitalPinToInterrupt()`.

Op een Leonardo zijn pin 2 en pin 3 net **SDA** en **SCL**, de twee draden van de I²C-bus. Op zo'n bord kan je die pinnen dus niet gebruiken voor een interrupt als je ook een PCF8574 aan het werk hebt. Op de UNO zit I²C op A4 en A5 en heb je dat probleem niet.

RISING, FALLING en CHANGE

Het derde onderdeel zegt op welke **verandering** je wil reageren. Dat is het moment waarop de spanning van laag naar hoog gaat of omgekeerd, en dus niet het niveau zelf. Zo'n overgang heet een **flank**.

Waarde	Gaat af wanneer	Gebruik je bij
FALLING	De pin gaat van HIGH naar LOW (dalende flank)	Een knop met <code>INPUT_PULLUP</code> : die is hoog in rust en wordt laag als je drukt. Dit is wat je in dit labo bijna altijd nodig hebt.
RISING	De pin gaat van LOW naar HIGH (stijgende flank)	Een knop met een externe pull-downweerstand, of een sensor die een puls omhoog geeft.
CHANGE	Bij allebei: elke verandering	Als je zowel het indrukken als het loslaten wil weten. Let op: je ISR gaat dan dus twee keer af per druk.

De drie soorten flanken

💡 MET INPUT_PULLUP HOORT FALLING

Gebruik je `pinMode(knopPin, INPUT_PULLUP)`, dan staat de pin hoog zolang je niets doet, en trekt de knop hem naar de massa als je drukt. De flank op het moment van indrukken is dus een **dalende** flank, en dat is **FALLING**.

Herlees zo nodig [Pull up en pull down weerstanden](#).

De volgorde in je setup()

Zet je `pinMode()` altijd **voor** je `attachInterrupt()`:

```
1  const int knopPin = 2;
2  const int ledPin = 8;
3
4  void mijnISR()
5  {
6      digitalWrite(ledPin, HIGH);
7  }
8
9  void setup()
10 {
11     pinMode(ledPin, OUTPUT);
12
13     pinMode(knopPin, INPUT_PULLUP);           // eerst dit
14     attachInterrupt(digitalPinToInterrupt(knopPin), mijnISR, FALLING); // dan pas dit
15 }
16
17 void loop()
18 {
19 }
```

CPP

Doe je het omgekeerd, dan schakel je de pull-up pas in nadat de interrupt al luistert. Die pin springt daardoor van laag naar hoog, en dat is een flank die je zelf gemaakt hebt. Je ISR kan daardoor meteen bij het opstarten één keer afgaan zonder dat er iemand op de knop gedrukt heeft.

Een interrupt weer uitzetten

Wil je op een bepaald moment niet meer gestoord worden, dan maak je de koppeling weer los:

```
detachInterrupt(digitalPinToInterrupt(knopPin));
```

CPP

Je hebt dit in dit labo niet nodig, maar het is handig om te weten dat het bestaat. Voor een noodstop wil je meestal juist het omgekeerde, namelijk dat hij altijd blijft luisteren.

Verder lezen

- [De regels van een ISR](#): wat mag er in die functie staan, en vooral wat niet.
- [volatile en de vlagvariabele](#): hoe je gegevens uit je ISR naar je `loop()` krijgt.

De regels van een ISR

Een interrupt service routine is een gewone functie die je zelf schrijft, maar er gelden strengere regels dan je gewoon bent. Overtreed je ze, dan krijg je zelden een foutmelding, maar wel een programma dat vastloopt of vreemd doet.

Hoe een ISR eruitziet

```
1 void noodstopIngedrukt()  
2 {  
3     digitalWrite(ledPin, LOW);  
4  
5     noodstop = true;  
6 }
```

CPP

Twee dingen liggen vast:

- Ze is altijd **void**. Een ISR kan niets teruggeven, want er is niemand die op een antwoord zit te wachten. Er is geen enkele regel in je programma die haar oproept.
- Ze heeft **geen parameters**. Tussen de haakjes staat niets, want er is ook niemand om iets mee te geven.

De naam mag je zelf kiezen. Kies er een die zegt wat er gebeurd is, niet wat de functie doet:

`noodstopIngedrukt`, `knopVeranderd`, `expanderMelddeIets`.

Regel één: zo kort mogelijk

Terwijl je ISR draait, ligt de rest van je programma stil. Niet alleen je `loop()` staat stil, ook alle andere interrupts staan uit. De chip zet ze automatisch uit zodra hij in een ISR springt, en pas weer aan als je eruit komt.

Dat is meteen de reden achter alle regels hieronder. Alles wat lang duurt in een ISR, houdt de hele controller tegen. En alles wat in een ISR op een andere interrupt zit te wachten, wacht voor eeuwig.

delay() blijft hangen

`delay()` werkt niet in een ISR, en dat op de vervelendst mogelijke manier:

```
1 void knopIngedrukt()  
2 {  
3     digitalWrite(ledPin, HIGH);  
4     delay(500);           // FOUT: je programma komt hier nooit meer uit  
5     digitalWrite(ledPin, LOW);  
6 }
```

CPP

`delay()` telt de tijd af met behulp van een timer die zelf een interrupt gebruikt. Die interrupt staat uit terwijl jouw ISR draait, dus de tijd staat stil, dus `delay()` is nooit klaar. Je bord hangt, en alleen de resetknop helpt nog.

⚠ WAAROM EEN KORTE DELAY WEL LIJKT TE WERKEN

Test je met `delay(1)`, dan komt je programma er wel uit en denk je dat het meevalt. Dat is toeval, want voor heel korte wachttijden kan de chip het nog net zonder die timer af. Vanaf een paar milliseconden hangt het gegarandeerd. Doe het dus niet.

Moet je in een ISR echt kort wachten, dan bestaat `delayMicroseconds()`, want die telt instructies. Maar als je in een ISR aan het wachten bent, klopt je ontwerp meestal niet.

Wire hangt je bord op

Dit is de gemeenste van allemaal, en je botst er in dit labo op zodra je met de PCF8574 werkt.

```
1 void expanderMelddeIets()  
2 {  
3     Wire.requestFrom(adresExpander, 1); // FOUT: je bord hangt hier vast  
4     byte knoppen = Wire.read();  
5 }
```

CPP

De I²C-hardware in de chip werkt byte per byte, en gebruikt **zelf een interrupt** om na elke byte verder te gaan. `Wire.requestFrom()` zet de boel in gang en blijft dan wachten tot die interrupt het werk afgemaakt heeft.

Maar je zit in een ISR. De interrupts staan uit. Die hulp komt dus nooit, en `Wire.requestFrom()` blijft eeuwig wachten. Je bord hangt tot je op reset drukt, en er is geen foutmelding en geen timeout.

Hetzelfde geldt voor `Wire.beginTransmission()` en `Wire.endTransmission()`. Kortom, je gebruikt geen enkele I²C-opdracht in een ISR.

⚠️ WAAROM INTERRUPTS() IN JE ISR GEVAARLIJK IS

Vroeg of laat bedenkt iemand dat je de interrupts toch weer kan aanzetten met `interrupts()` als eerste regel van je ISR.

Dat werkt, en daarom is het gevaarlijk. Zodra de interrupts weer aanstaan, mag je knop je ISR opnieuw onderbreken, terwijl de eerste nog bezig is. Een denderende knop doet dat tientallen keren na elkaar, en elke onderbreking legt een nieuwe laag op de stapel. Je Arduino heeft 2 kB geheugen. Die stapel loopt over, je variabelen worden overschreven en je programma doet vanaf dan volstrekt willekeurige dingen. Dat is veel moeilijker te vinden dan een bord dat hangt.

De oplossing is het werk uit de ISR te halen met [het vlagpatroon](#).

Serial vertraagt alles

Printen vanuit een ISR is verboden, maar om een andere reden dan `Wire`. Je bord hangt er niet van, en daardoor lijkt het te werken wanneer je het probeert.

Bij 9600 baud duurt één teken ruim een milliseconde. Normaal geeft je Arduino de tekens beetje bij beetje weg terwijl je programma verder loopt. In een ISR kan dat niet, dus zit je ISR ineens tientallen milliseconden vast te schrijven. Al die tijd staat alles stil: je mist andere knopdrukken, je klok loopt achter, en een I²C-overdracht die elders bezig was gaat stuk.

En omgekeerd heeft `Serial.available()` in een ISR al helemaal geen zin, want binnenkomende tekens worden zelf met een interrupt opgevangen, dus terwijl jouw ISR draait kan er onmogelijk iets nieuws binnenkomen.

💡 EÉN REGEL VOOR WIRE EN SERIAL SAMEN

De gevolgen verschillen, maar de regel is dezelfde. Doe geen in- of uitvoer in een ISR, maar zet een vlag en laat je `loop()` het printen doen.

millis() staat stil, maar je mag hem wel lezen

De klok van `millis()` wordt zelf bijgewerkt door een interrupt. Die staat uit tijdens je ISR, dus **de tijd loopt niet verder** zolang je erin zit. Roep je `millis()` twee keer op in dezelfde ISR, dan krijg je twee keer hetzelfde getal.

Je mag hem wel degelijk lezen. Je krijgt de waarde van vlak voor de onderbreking, en bij de volgende keer dat je ISR afgaat staat de klok gewoon weer verder, want tussendoor heeft je `loop()` gedraaid. Voor iets als "is het al 200 ms geleden?" is dat ruim nauwkeurig genoeg, en daar maak je in [Denderen bij een interrupt](#) gebruik van.

Wat mag dan wel?

Wat	Waarom het mag
Een <code>volatile</code> variabele veranderen	Een paar instructies, klaar. Dit is de kern van het vlagpatroon.
<code>digitalWrite()</code> en <code>digitalRead()</code>	Die praten rechtstreeks met de pin en hebben geen andere interrupt nodig. Een paar microseconden werk.
<code>millis()</code> lezen	Levert een geldige waarde op, ook al loopt de klok even niet verder.
Rekenen met gewone variabelen	Mag, zolang het kort is. Een <code>for</code> -lus van vijf rondes kan, een lus van duizend niet.

Toegelaten in een ISR

Samengevat

DE REGELS OP EEN RIJTJE

- Altijd `void`, altijd zonder parameters.
- Zo kort mogelijk. Idealiter merkt niemand dat je programma onderbroken werd.
- Geen `delay()`, geen `Wire`, geen `Serial`.
- `digitalWrite()` mag, en is vaak wat je nodig hebt.
- Alles wat je met de buitenwereld deelt, is `volatile`.
- Moet er meer gebeuren, zet dan een vlag en laat je `loop()` het afmaken.

volatile en de vlagvariabele

Je ISR moet kort blijven, maar er moet wel iets gebeuren. De oplossing is dat je ISR alleen een boodschap achterlaat en je `loop()` het echte werk doet. Om die boodschap onderweg niet kwijt te spelen, heb je het woord `volatile` nodig.

Waarom volatile

Een variabele die je zowel in je ISR als in je `loop()` gebruikt, moet `volatile` zijn:

```
volatile bool knopGedrukt = false;
```

CPP

De reden zit in de compiler. Wanneer die ziet dat je in je `loop()` steeds opnieuw dezelfde variabele uitleest, houdt hij die waarde bij in de processor zelf in plaats van hem telkens uit het geheugen te halen, omdat dat sneller is. Vaak is dat de juiste beslissing.

Maar jouw variabele verandert wel degelijk, namelijk in je ISR. En die ISR staat nergens in je `loop()`, dus de compiler ziet het verband niet. Het gevolg is een `loop()` die een verouderde kopie blijft lezen en nooit merkt dat de knop ingedrukt is.

Met `volatile` zeg je dat de compiler die variabele elke keer opnieuw uit het geheugen moet halen, omdat hij niet kan weten wanneer ze verandert.

DE COMPILER WAARSCHUWT HIER NOOIT VOOR

Vergeet je `volatile`, dan compileert alles. Erger nog is dat het vaak ook werkt, tot je iets kleins verandert en de compiler ineens besluit om wel een kopie bij te houden. Dan lijkt het alsof een onschuldige aanpassing je programma stukgemaakt heeft.

Elke variabele die je in een ISR aanraakt, krijgt `volatile`, zonder uitzondering.

Het vlagpatroon

Dit is het belangrijkste patroon van het hele labo, en je gaat het in bijna elke oefening gebruiken.

Je ISR doet niets anders dan een `bool` op `true` zetten. Zo'n variabele heet een **vlag**. Je `loop()` kijkt of die vlag omhoog staat, doet dan het echte werk, en zet ze weer omlaag.

```

1  const int knopPin = 2;
2  const int ledPin = 8;
3
4  volatile bool knopGedrukt = false;
5
6  bool ledBrandt = false;
7
8  // De ISR: zo kort als het maar kan
9  void knopISR()
10 {
11     knopGedrukt = true;
12 }
13
14 void setup()
15 {
16     pinMode(ledPin, OUTPUT);
17     pinMode(knopPin, INPUT_PULLUP);
18
19     Serial.begin(9600);
20
21     attachInterrupt(digitalPinToInterrupt(knopPin), knopISR, FALLING);
22 }
23
24 void loop()
25 {
26     if (knopGedrukt == true)
27     {
28         knopGedrukt = false; // eerst de vlag neerhalen
29
30         ledBrandt = !ledBrandt;
31         digitalWrite(ledPin, ledBrandt);
32
33         Serial.println("Er is gedrukt"); // dit mag hier wel, in de ISR niet
34     }
35 }

```

Kijk wat je hiermee wint. Het printen, dat in een ISR verboden is, staat nu in de `loop()` en mag dus. Hetzelfde geldt voor een `Wire`-opdracht, een `delay()`, of eender wat dat te lang duurt. Je ISR blijft één regel.

HAAL DE VLAG NEER VÓÓR JE AAN HET WERK GAAT

Zet `knopGedrukt = false;` bovenaan, niet onderaan. Doe je het op het einde, dan wis je ook een druk die net tijdens je werk binnenkwam, en die druk is dan voorgoed weg.

Zet je ze bovenaan en komt er tijdens je werk een nieuwe druk binnen, dan zet de ISR de vlag opnieuw en handel je die de volgende ronde af. In het slechtste geval doe je één keer overbodig werk. Dat is veel beter dan iets missen.

Een getal van vier bytes heeft meer nodig

Bij een `bool` of een `byte` hoef je verder niets te doen. De processor haalt zo'n waarde met één instructie uit het geheugen, en een instructie kan niet halverwege onderbroken worden.

Bij een `int` (twee bytes) of een `long` (vier bytes) ligt dat anders. Die haalt de processor stuk voor stuk op, en tussen die stukken door kan je ISR toeslaan.

⚠ EEN GETAL DAT NOOIT BESTAAN HEEFT

Stel: je teller staat op 255 en je ISR telt hem op naar 256. In het geheugen zijn dat twee bytes.

1. Je `loop()` leest de onderste byte: 255.
2. De ISR slaat toe en telt op. De onderste byte wordt 0, de bovenste wordt 1.
3. Je `loop()` leest de bovenste byte: 1.

Je `loop()` plakt die twee samen tot 511, een getal dat op geen enkel moment in je programma bestaan heeft. Het gebeurt maar zelden, dus je vindt het pas na een halve dag zoeken.

De oplossing is de interrupts kort uitzetten terwijl je het getal ophaalt:

```

1  const int knopPin = 2;
2
3  volatile unsigned long aantalDrukken = 0;
4
5  unsigned long vorigAantal = 0;
6
7  void knopISR()
8  {
9      aantalDrukken++;
10 }
11
12 void setup()
13 {
14     pinMode(knopPin, INPUT_PULLUP);
15
16     Serial.begin(9600);
17
18     attachInterrupt(digitalPinToInterrupt(knopPin), knopISR, FALLING);
19 }
20
21 void loop()
22 {
23     noInterrupts();                // even niet storen
24     unsigned long aantal = aantalDrukken; // vier bytes in een keer overnemen
25     interrupts();                  // en meteen weer aan
26
27     if (aantal != vorigAantal)
28     {
29         vorigAantal = aantal;
30
31         Serial.println(aantal);
32     }
33 }

```

Je maakt dus een **kopie**, en werkt daarna verder met die kopie. Tussen `noInterrupts()` en `interrupts()` staan zo weinig mogelijk regels. Zet je er je `Serial.println()` ook nog tussen, dan maak je net het probleem dat je wou vermijden.

i DRUKKEN TIJDENS NOINTERRUPTS() BLIJVEN BEWAARD

Drukt iemand op de knop terwijl de interrupts uitstaan, dan onthoudt de hardware dat. Zodra `interrupts()` uitgevoerd is, gaat je ISR alsnog af. Je verliest hoogstens een paar microseconden.

Eén vuistregel om alles simpel te houden

ALLEEN DE ISR SCHRIJFT, DE LOOP LEEST

Zorg dat je gedeelde variabelen maar op één plaats aangepast worden. De ISR telt op, de `loop()` kijkt alleen. Ga je in je `loop()` ook `aantalDrukken++` schrijven, dan heb je hetzelfde probleem opnieuw, want optellen is in werkelijkheid drie stappen: ophalen, plus één, terugzetten. En daar kan de ISR net zo goed tussen springen.

De enige uitzondering is een vlag die je in de `loop()` neerhaalt. Dat is één byte, dus één instructie, en dus veilig.

Verder lezen

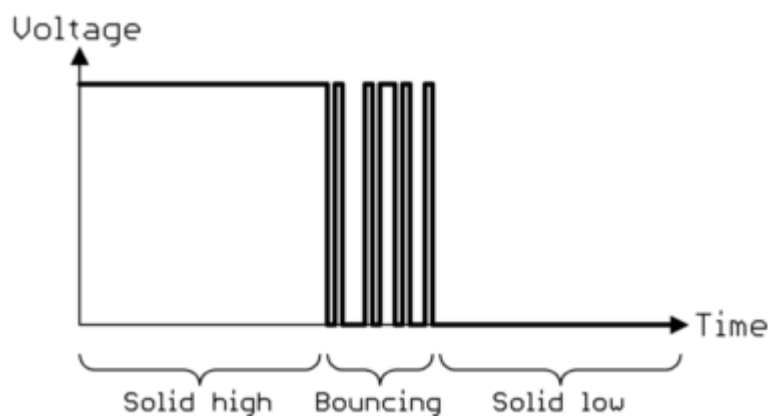
- [Denderen bij een interrupt](#): waarom je teller sneller loopt dan je drukt.
- [De regels van een ISR](#): wat je uit die ISR moet houden.

Denderen bij een interrupt

Je bouwt een teller die optelt bij elke druk op de knop. Je drukt één keer, en de teller springt naar 6. Je code klopt nochtans. Wat er gebeurt is dendering, en bij een interrupt is ze veel vervelender dan bij pollen.

Wat er gebeurt

Een drukknop is een stuk veerend metaal. Duw je erop, dan stuitert hij een paar keer heen en weer voor hij tot rust komt. Dat duurt maar een paar milliseconden, maar in die tijd gaat de spanning op je pin vijf tot twintig keer op en neer.



☞ Wat je knop echt doet bij één druk. Elke dalende flank in die rommel is er een waar je interrupt op afgaat.

Je kent dit al uit [Debouncen](#). Bij pollen viel het nog mee, want je `loop()` kijkt maar af en toe, en met een korte `delay()` na een verandering was je ervan af.

Bij een interrupt ligt dat anders. De hardware mist geen enkele flank, dat is nu net waar je hem voor gekozen hebt. Elk van die stuiteringen is een dalende flank, en dus gaat je ISR er **allemaal** een keer voor af.

⚠ HERKEN JE DIT?

- Je teller springt met sprongen van 3 tot 10 tegelijk.
- Je led die moet omschakelen, doet soms wel en soms niets. (Logisch: bij een even aantal flanken sta je weer waar je begon.)
- Je stuurt één boodschap over de seriële lijn en er komen er zeven aan.

Dat is dendering. Je code is niet fout, je knop is geen ideale schakelaar.

De oplossing: een vergrendeling

Je kan het stuiten niet tegenhouden, maar je kan het wel negeren. Onthoud wanneer je de vorige druk aanvaard hebt, en negeer alles wat binnen de volgende 200 milliseconden nog binnenkomt.

Een mens kan onmogelijk vijf keer per seconde bewust drukken, dus je verliest niets. Het gestuiter is na een paar milliseconden voorbij, dus je vangt alles op.

i WAAROM HIER 200 MS EN BIJ HET POLLEN 50 MS

Bij het pollen vergelijk je de huidige stand met de vorige. Indrukken is HIGH naar LOW, loslaten is LOW naar HIGH, en dat zijn twee verschillende overgangen. Je `delay()` hoeft daar dus alleen het gestuiter zelf te overleven, en 50 ms is daarvoor ruim genoeg.

Hier telt je vergrendeling vanaf het *indrukken*, terwijl ook het loslaten dalende flanken oplevert. Ze moet dus de hele beweging overbruggen, en een gewone tik duurt al snel meer dan 100 ms. Vandaar het hogere getal.

De twee constanten heten daarom anders. Bij het pollen gebruik je `ontdenderTijd` van 50 ms, de constante die je uit `Debouncen` kent. Bij een interrupt gebruik je `vergrendelTijd` van 200 ms.

```
1  const int knopPin = 2;
2  const int ledPin = 8;
3
4  const unsigned long vergrendelTijd = 200; // zolang negeren we nieuwe flanken
5
6  volatile bool ledBrandt = false;
7  volatile unsigned long laatsteDruk = 0;
8
9  void knopIngedrukt()
10 {
11     unsigned long nu = millis();
12
13     if (nu - laatsteDruk < vergrendelTijd)
14     {
15         return; // dit is gestuiter, geen nieuwe druk
16     }
17
18     laatsteDruk = nu;
19
20     ledBrandt = !ledBrandt;
21     digitalWrite(ledPin, ledBrandt);
22 }
23
24 void setup()
25 {
26     pinMode(ledPin, OUTPUT);
27     pinMode(knopPin, INPUT_PULLUP);
28
29     attachInterrupt(digitalPinToInterrupt(knopPin), knopIngedrukt, FALLING);
30 }
31
32 void loop()
33 {
34 }
```

De `return` stapt meteen uit de ISR, en dat is de goedkoopste manier om een flank weg te gooien.

Mag millis() dan wel in een ISR?

In [De regels van een ISR](#) staat dat de klok van `millis()` stilstaat zolang je in een ISR zit, en dat klopt.

Maar hier hoeft die klok niet verder te lopen. Je wil alleen weten hoe laat het was op het moment dat je onderbroken werd, en dat is wat je krijgt. Tussen twee knopdrukken in draait je `loop()`, dus tegen de tijd dat je ISR opnieuw afgaat staat de klok weer verder.

Voor een vergrendeling van 200 ms is dat ruim nauwkeurig genoeg. Je zou hem alleen niet moeten gebruiken om binnen één ISR een tijdsduur te meten, want daar staat hij inderdaad stil.

Waar deze oplossing nog tekortschiet

Er is één geval waarin de vergrendeling het laat afweten, namelijk wanneer je de knop **lang ingedrukt houdt**.

Ook bij het loslaten stuitert een knop. Houd je hem een seconde vast, dan is de eerste flank van dat losgelaten-gestuiter meer dan 200 ms na je druk, en wordt hij dus aanvaard als een tweede druk. Je led schakelt terug om.

In de praktijk valt dat vaak mee, omdat de meeste knoppen bij het loslaten veel minder stuiteren dan bij het indrukken. Merk je het toch, dan zijn er twee wegen:

- De vergrendeling langer maken, bijvoorbeeld 500 ms. Dat is snel gedaan, maar dan reageert je knop trager.
- De interrupt alleen gebruiken om je `loop()` wakker te maken, en de eigenlijke ontdendering met de toestandsmethode uit `Debouncen` in je `loop()` doen. Meer werk, maar volledig sluitend.

IN TINKERCAD ZIE JE HIER NIETS VAN

De drukknop in TinkerCAD maakt precies één keer contact en stuitert nooit. Je vergrendeling lijkt daar dus overbodige code die niets doet.

Laat ze staan. Op een echt bord verdient ze zich meteen terug, en dat je het probleem in de simulator niet ziet, maakt het net gevaarlijker, want je zou anders pas op het labo ontdekken dat je teller nergens op slaat.

Samengevat

ONTDENDEREN BIJ EEN INTERRUPT

- Elke stuitering is een flank, en dus een oproep van je ISR.
- Onthou met `millis()` wanneer je de vorige druk aanvaardde.
- Komt er binnen de 200 ms een nieuwe, stap dan meteen uit de ISR met `return`.
- Die tijdsvariabele is `volatile`, net als al de rest die je ISR aanraakt.
- Test op echte hardware, niet alleen in TinkerCAD.